Министерство образования и науки Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 Математика и компьютерные науки

Отчет о выполнении лабораторной работы №2 по дисциплине «Алгоритмические основы компьютерной графики»

«Алгоритм Вейлера-Айзертона удаления невидимых линий»

Выполнила студентка группы
№5130201/30101 Лаишевкина А. М. _____
Проверил Курочкин М. А. _____

«____»_____ 2026г.

Санкт-Петербург, 2026

Содержание

1 Введение **3**

2 Постановка задачи **4**

3 Описание метода **5**
3.1 Общий принцип 5
3.2 Предварительная сортировка по глубине 5
3.3 Отсечение многоугольников 5
3.4 Сравнение глубин и удаление экранированных многоугольников 6
3.5 Рекурсивное разбиение плоскости 7
3.6 Обработка случаев циклического перекрытия многоугольников 8

4 Описание реализации разработанного алгоритма **9**
4.1 Представление геометрических объектов 9
4.2 Класс WeilerAtherton 9
4.3 Удаление невидимых линий 12

5 Результаты работы алгоритма **14**

6 Сравнение с библиотечной реализацией **16**
6.1 Методика тестирования 16
6.2 Результаты сравнения 16
6.3 Анализ результатов 17

7 Заключение **18**

1 Введение

Компьютерная графика – область информатики, ориентированная на создание, хранение, редактирование и отображение изображений и трёхмерных моделей. Она используется в различных сферах, включая игры, кино, виртуальную и дополненную реальность, а также в научных и инженерных приложениях.

Задача удаления невидимых линий и поверхностей является одной из наиболее сложных в компьютерной графике. Алгоритмы удаления невидимых линий и поверхностей служат для определения линий ребер, поверхностей или объемов, которые видимы или невидимы для наблюдателя, находящегося в заданной точке пространства.

В данной лабораторной работе реализован алгоритм Вейлера-Айзертона удаления невидимых линий. Алгоритм предназначен для определения видимости линий в трехмерной сцене и удаления тех, которые находятся за другими объектами. Программа написана на языке C++. Также проводится сравнение собственной реализации с библиотечной реализацией операции пересечения многоугольников.

2 Постановка задачи

Цель работы: изучить и реализовать алгоритм Вейлера-Айзертона, а также сравнить собственную реализацию с библиотечным аналогом по производительности.

Задачи работы:

1. Изучить алгоритм Вейлера-Айзертона удаления невидимых линий.
2. Реализовать алгоритм на выбранном языке программирования.
3. Провести тестирование и сравнение с библиотечным аналогом по критерию времени выполнения.

Формальная постановка задачи.

Дано: набор геометрических тел, заданных многоугольными гранями, точка наблюдения и плоскость проекции (экран).

Найти: изображение сцены на плоскости проекции с удалением невидимых линий и поверхностей.

Алгоритмическая сложность: $O(n^2)$, где n – число многоугольников.

3 Описание метода

3.1 Общий принцип

Алгоритм Вейлера-Айзертона предназначен для уменьшения количества шагов разбиения, работая путем разбиения окна вдоль границ многоугольников. Выходными данными этого алгоритма, который для достижения необходимой точности работает в пространстве объекта, служат многоугольники.

Шаги:

1. Предварительная сортировка по глубине.
2. Отсечение по границе ближайшего к наблюдателю многоугольника, называемое сортировкой многоугольников на плоскости.
3. Удаление многоугольников, экранированных многоугольником, ближайшим к точке наблюдения.
4. Если требуется, то рекурсивное подразбиение и окончательная сортировка для устранения всех неопределенностей.

Особенности:

- Работает в пространстве объекта.
- Применим для удаления как невидимых линий, так и поверхностей.
- Использует списки приоритетов и рекурсивное разбиение плоскости.

3.2 Предварительная сортировка по глубине

Предварительная сортировка по глубине нужна для формирования списка приблизительных приоритетов. Предположим, что точка наблюдения расположена в бесконечности на положительной полуоси z, тогда ближайшим к ней и первым в списке будет тот многоугольник, который обладает вершиной с максимальной координатой z.

3.3 Отсечение многоугольников

В качестве отсекающего многоугольника используется копия первого многоугольника из предварительного списка приоритетов по глубине. Отсекаться будут остающиеся в этом списке многоугольники, включая и первый многоугольник. Вводятся два списка: внутренний и внешний. С помощью алгоритма отсечения Вейлера-Азертона все многоугольники отсекаются по границам

отсекающего многоугольника. Фактически это двумерная операция отсечения проекций отсекающего и отсекаемого многоугольников. Та часть каждого отсекаемого многоугольника, которая оказывается внутри отсекающего, если она имеется, попадает во внутренний список. Оставшаяся часть, если таковая есть, попадает во внешний список. Этот этап алгоритма является сортировкой на плоскости или xy-сортировкой. Пример приведен на рис. 1. На рис. 2 показаны внутренний и внешний списки для сцены на рис. 1.

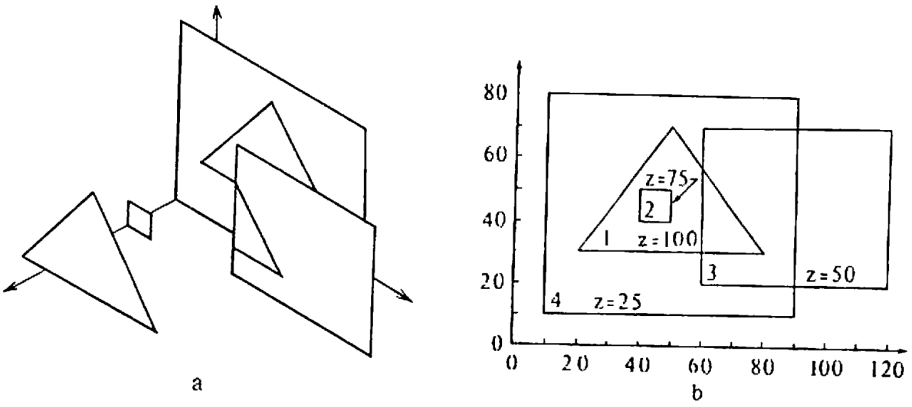


Рис. 1. Отсечение многоугольников по приоритетам

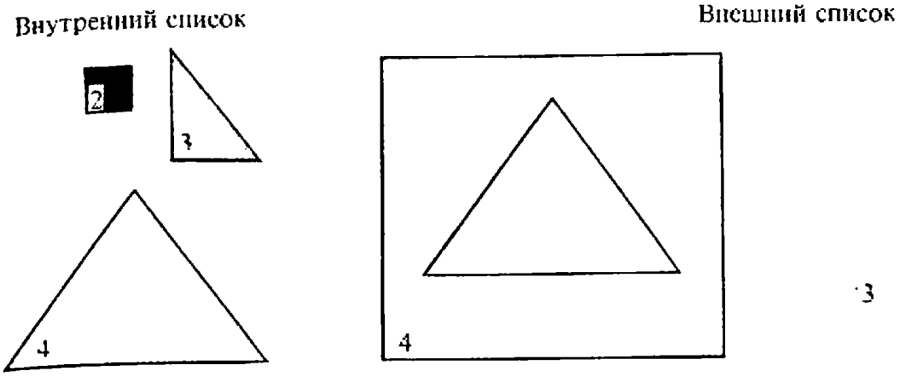


Рис. 2. Внутренний и внешний списки многоугольников

3.4 Сравнение глубин и удаление экранированных многоугольников

Теперь сравниваются глубины каждого многоугольника из внутреннего списка с глубиной отсекающего многоугольника. С использованием координат (x, y) вершин отсекаемых многоугольников и уравнений несущих плоскостей вычисляются глубины (координаты z) каждой вершины. Затем они сравниваются с минимальной координатой $z(z_{cmin})$ для отсекающего многоугольника. Если глубина ни одной из этих вершин не будет больше z_{cmin}, то

все многоугольники из внутреннего списка экранируются отсекающим многоугольником (рис. 1). Эти многоугольники удаляются, и изображается внутренний список. Заметим, что во внутреннем списке остался лишь отсекающий многоугольник. Работа алгоритма затем продолжается с внешним списком.

3.5 Рекурсивное разбиение плоскости

Если координата z какого-либо многоугольника из внутреннего списка окажется больше, чем z_{cmin}, то такой многоугольник по крайней мере частично экранирует отсекающий многоугольник. На рис. 3 показано, как это может произойти. В подобном случае результат предварительной сортировки по глубине ошибочен. Поэтому алгоритм рекурсивно подразделяет плоскость (x, y), используя многоугольник, нарушивший порядок, в качестве нового отсекающего многоугольника. Отсечению подлежат многоугольники из внутреннего списка, причем старый отсекающий многоугольник теперь сам будет подвергнут отсечению новым отсекающим многоугольником. Подчеркнем, что новый отсекающий многоугольник является копией исходного многоугольника, а не его остатка после первого отсечения. Использование копии неотсеченного многоугольника позволяет минимизировать число разбиений.

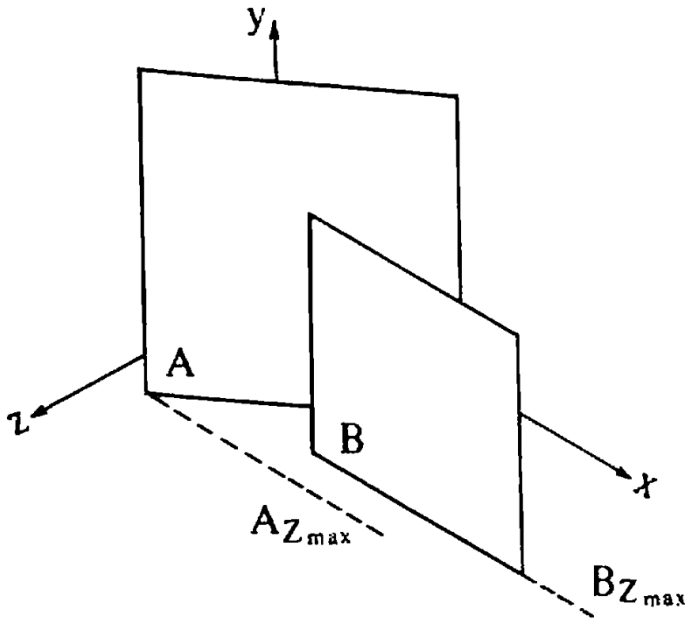


Рис. 3. Условие возникновения ошибочного результата в предварительной сортировке по глубине

Алгоритм заканчивает работу после обработки всех рёбер и определения для каждого из них видимых и невидимых фрагментов. После обработки всех многоугольников формируется окончательный результат удаления невидимых линий.

3.6 Обработка случаев циклического перекрытия многоугольников

Заслуживает внимание еще одна дополнительная деталь этого алгоритма. Когда некоторый многоугольник циклически перекрывается с отсекающим, т.е. когда он лежит как впереди, так и позади отсекающего (рис. 4, a), то в рекурсивном разбиении необходимости нет. Дело в том, что все экранируемое циклическим многоугольником уже было удалено на предыдущем шаге отсечения. Необходимо лишь произвести отсечение исходного многоугольника по границам циклического многоугольника и изобразить результат. Ненужное рекурсивное разбиение можно предотвратить, если создать список многоугольников, которые уже использовались как отсекающие. Тогда, если при рекурсивном разбиении текущий отсекающий многоугольник появляется в этом списке, значит, обнаружен циклически перекрывающийся многоугольник. Следовательно, не требуется никакого дополнительного разбиения. Заметим, что данный алгоритм непосредственно обрабатывает случаи циклического перекрытия сразу нескольких многоугольников, как показано на рис. 4, b.

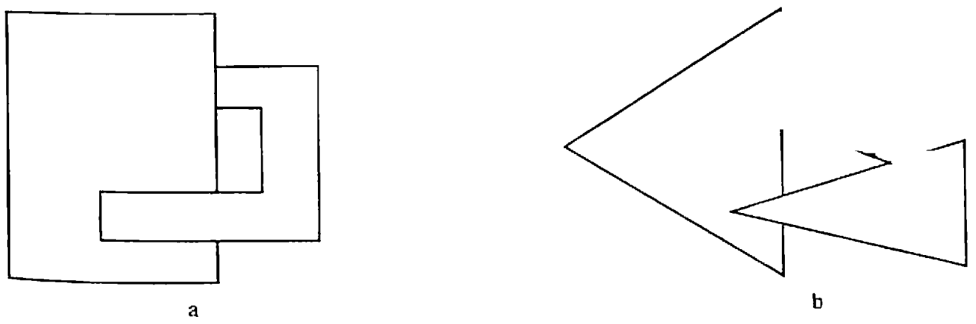


Рис. 4. Циклически перекрывающиеся многоугольники

4 Описание реализации разработанного алгоритма

4.1 Представление геометрических объектов

Для хранения элементов сцены в программе используются структуры `Point2`, `Point3` и `Polygon3`. Двумерная точка применяется при обработке проекций многоугольников, трёхмерная — для хранения исходных координат объектов сцены. Многоугольник представлен вектором его вершин.

Листинг 1. Структуры для представления точек и многоугольников

```
struct Point2 { double x{}, y{}; };
struct Point3 { double x{}, y{}, z{}; };
struct Polygon3 { std::vector<Point3> v; };
```

В процессе удаления невидимых линий исходные трёхмерные многоугольники преобразуются в их проекции на плоскость (xy). Для этого используется функция `projectXY()`.

Листинг 2. Проецирование многоугольника на плоскость наблюдения

```
static Polygon2 projectXY(const Polygon3& p) {
    Polygon2 out;
    out.reserve(p.v.size());

    for (const auto& q : p.v)
        out.push_back({q.x, q.y});

    return out;
}
```

4.2 Класс WeilerAtherton

Основная часть собственной реализации алгоритма вынесена в класс `WeilerAthert`. Его метод `intersect()` принимает два двумерных многоугольника и возвращает набор многоугольников, соответствующих их пересечению.

Внутри класса используется структура `Node`, описывающая вершину циклического списка. Помимо координат точки, узел хранит признаки точки пересечения, входа в другой многоугольник, посещения и ссылки на соседние узлы.

Листинг 3. Узел циклического списка класса WeilerAtherton

```
struct Node {
    Point2 p{};
    bool intersection = false;
    bool entry = false;
    bool visited = false;

    Node* next = nullptr;
    Node* prev = nullptr;
    Node* neighbor = nullptr;
};
```

Структура `List` используется для формирования циклических списков вершин двух исходных многоугольников. Для каждой вершины задаются ссылки на следующую и предыдущую вершины. Дополнительный метод `insertAfter()` позволяет добавлять в список точки пересечения рёбер.

Листинг 4. Формирование и изменение списка вершин

```
struct List {
    std::vector<Node> nodes;
    Node* head = nullptr;

    explicit List(const Polygon2& p) {
        nodes.resize(p.size());

        for (size_t i = 0; i < p.size(); ++i)
            nodes[i].p = p[i];

        for (size_t i = 0; i < p.size(); ++i) {
            nodes[i].next = &nodes[(i + 1) % p.size()];
            nodes[i].prev = &nodes[(i + p.size() - 1) % p.size()];
        }

        head = nodes.empty() ? nullptr : &nodes[0];
    }

    void insertAfter(Node* pos, std::unique_ptr<Node> node) {
        Node* raw = node.get();
        owned.push_back(std::move(node));

        raw->prev = pos;
        raw->next = pos->next;
        pos->next->prev = raw;
        pos->next = raw;
    }

    std::vector<std::unique_ptr<Node>> owned;
};
```

Метод `intersect()` выполняет основные этапы алгоритма. Сначала для каждого ребра первого многоугольника проверяются все рёбра второго многоугольника и определяются точки их пересечения.

Листинг 5. Поиск точек пересечения рёбер

```
for (size_t i = 0; i < subject.size(); ++i) {
    const Point2 a1 = subject[i];
    const Point2 a2 = subject[(i + 1) % subject.size()];

    for (size_t j = 0; j < clip.size(); ++j) {
        const Point2 b1 = clip[j];
        const Point2 b2 = clip[(j + 1) % clip.size()];

        Intersection ix;

        if (segmentIntersection(a1, a2, b1, b2, ix)) {
            bool duplicate = false;

            for (const auto& old : xs) {
                if (samePoint(old.p, ix.p)) {
                    duplicate = true;
```

```
                    break;
                }
            }

            if (!duplicate)
                xs.push_back({i, j, ix.ta, ix.tb, ix.p});
        }
    }
}
```

После определения пересечения найденные точки вставляются в списки вершин обоих многоугольников. Затем для каждой точки определяется, является ли она входной или выходной.

Листинг 6. Определение входных и выходных точек

```
auto markEntryExit = [&](Node* start, const Polygon2& other) {
    Node* cur = start;

    do {
        if (cur->intersection) {
            const Point2 dir = cur->next->p - cur->p;
            const double len = std::sqrt(std::max(dist2(cur->p, cur->next->p
               ), 1e-24));

            const Point2 probe = cur->p + dir * (1e-7 / len);

            cur->entry = pointInPolygon(probe, other);
        }

        cur = cur->next;
    } while (cur != start);
};
```

Заключительным этапом `intersect()` является обход полученных циклических списков. При достижении выходной точки выполняется переход в список второго многоугольника через указатель `neighbor`. В результате формируется контур области пересечения.

Листинг 7. Построение результирующих многоугольников

```
for (Node* start : intersectionsA) {
    if (start->visited || !start->entry)
        continue;

    Polygon2 poly;
    Node* cur = start;
    size_t guard = 0;

    while (guard++ < (intersectionsA.size() + intersectionsB.size() + 10) *
       20) {
        cur->visited = true;
        poly.push_back(cur->p);

        cur = cur->next;

        if (cur == start)
            break;

        if (cur->intersection && !cur->entry) {
```

```
            Node* other = cur->neighbor;

            if (!other)
                break;

            if (other == start)
                break;

            other->visited = true;
            cur = other;

            poly.push_back(cur->p);
            cur = cur->next;
        }
    }

    dedupePolygon(poly);

    if (poly.size() >= 3 && std::abs(areaSigned(poly)) > 1e-10) {
        poly = ccw(poly);
        result.push_back(poly);
    }
}
```

4.3 Удаление невидимых линий

За удаление невидимых частей рёбер отвечает функция `removeHidden()`. Она является общей для собственной и библиотечной реализации: различие между ними задаётся передаваемой функцией пересечения многоугольников.

В начале работы многоугольники сцены сортируются по максимальной координате (z). Это позволяет сначала обрабатывать объекты, находящиеся ближе к наблюдателю.

Листинг 8. Сортировка многоугольников по глубине

```
std::vector<int> order(scene.size());

std::iota(order.begin(), order.end(), 0);

std::sort(
    order.begin(),
    order.end(),
    [&](int a, int b) {
        return maxZ(scene[a]) > maxZ(scene[b]);
    }
);
```

Затем каждое ребро многоугольника разбивается на части в местах пересечения с областями перекрытия. Для этого используется функция `splitByPolygon()`, возвращающая параметры точек разбиения на исходном отрезке.

После получения всех точек разбиения каждый фрагмент проверяется по его средней точке. Если средняя точка находится внутри области перекрытия, дополнительно сравнивается глубина поверхности и глубина ребра.

Листинг 9. Проверка видимости фрагмента ребра

```
for (size_t k = 0; k + 1 < ts.size(); ++k) {
    const double t0 = ts[k];
    const double t1 = ts[k + 1];

    if (t1 - t0 < 1e-10)
        continue;

    const double tm = (t0 + t1) * 0.5;

    const Point3 mid3 = lerp3(A3, B3, tm);
    const Point2 mid2{mid3.x, mid3.y};

    bool hidden = false;

    for (const auto& [oidx, mask] : masks) {
        if (!pointInPolygon(mid2, mask))
            continue;

        const double zo = zAt(scene[oidx], mid3.x, mid3.y);

        if (std::isfinite(zo) && zo > mid3.z + 1e-8) {
            hidden = true;
            break;
        }
    }

    if (!hidden) {
        result.push_back({
            lerp3(A3, B3, t0),
            lerp3(A3, B3, t1),
            sidx,
            static_cast<int>(e)
        });
    }
}
```

Если более близкий многоугольник закрывает текущий фрагмент, переменная `hidden` получает значение `true` и фрагмент не добавляется в результирующий набор. В противном случае он сохраняется как видимый.

5 Результаты работы алгоритма

Для проверки корректности алгоритма была сформирована трёхмерная сцена из трёх плоских четырёхугольников. Многоугольники имеют различные значения координаты z, а их проекции частично перекрываются. Это позволяет проверить как полностью видимые рёбра, так и рёбра, видимые только частично.

На рисунке 1 представлена проекция сцены №1 до удаления невидимых линий.

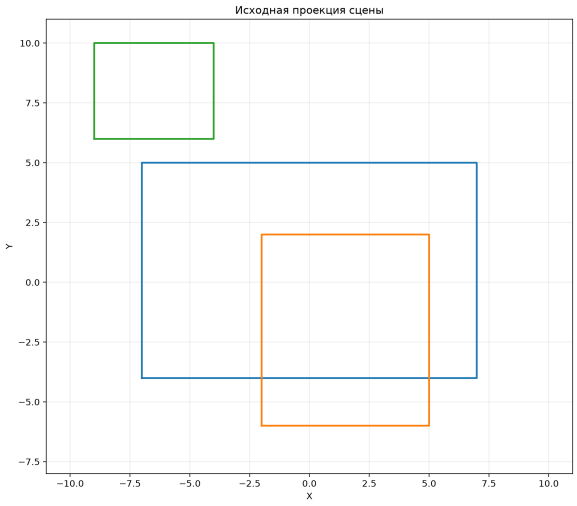


Рис. 5. Пример 1: до применения алгоритма

На рисунке 2 представлена проекция сцены №1 после удаления невидимых линий.

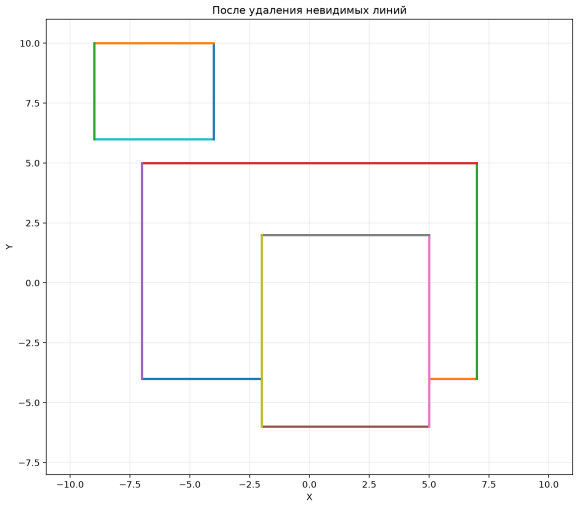


Рис. 6. Пример 1: после применения алгоритма

На рисунке 3 представлена проекция сцены №2 до удаления невидимых линий.

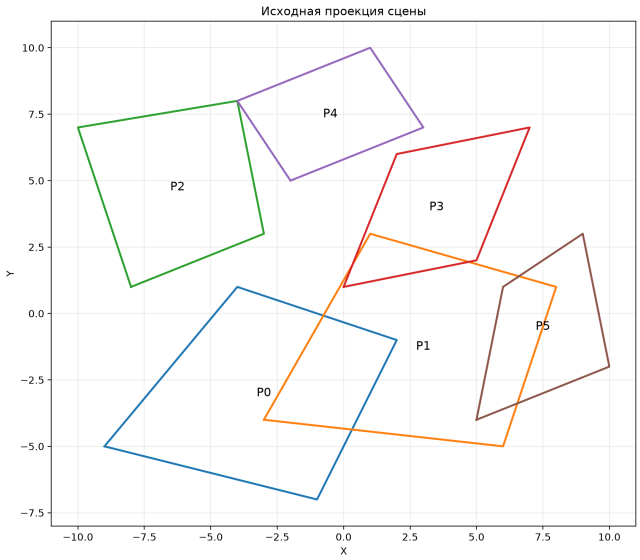


Рис. 7. Пример 2: до применения алгоритма

На рисунке 4 представлена проекция сцены №2 после удаления невидимых линий.

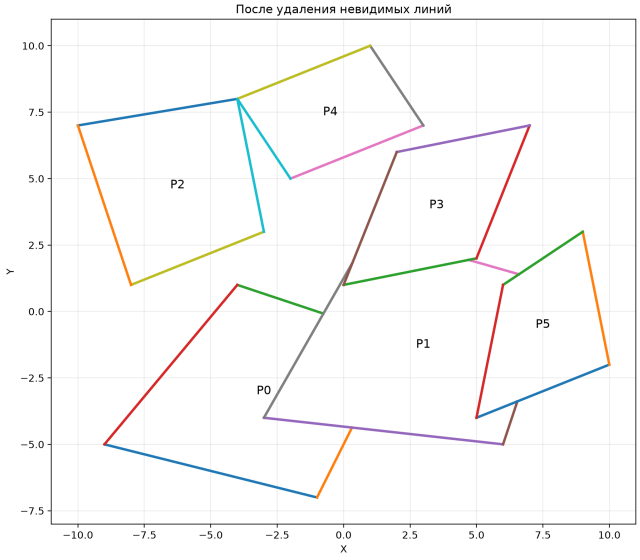


Рис. 8. Пример 2: после применения алгоритма

6 Сравнение с библиотечной реализацией

Для сравнения собственной реализации алгоритма Вейлера–Айзертона с библиотечной реализацией использовалась библиотека Boost.Geometry. В качестве библиотечного аналога применялась функция пересечения многоугольников. Сравнение проводилось по критерию времени выполнения для различного количества многоугольников.

6.1 Методика тестирования

Для сравнения использовались следующие параметры:

- Количество многоугольников: 10, 20, 40, 80.
- Каждый замер выполнялся 10 раз, в таблице приведено среднее время.

6.2 Результаты сравнения

В таблице 1 приведены результаты замеров.

Таблица 1. Результаты сравнения реализаций

№	Число многоугольников	Собственная реализация, мс	Библиотечная реализация, мс
1	10	0.254	0.319
2	20	1.083	1.134
3	40	3.248	2.890
4	80	10.140	7.319

График зависимости времени работы от количества многоугольников представлен на рисунке 5.

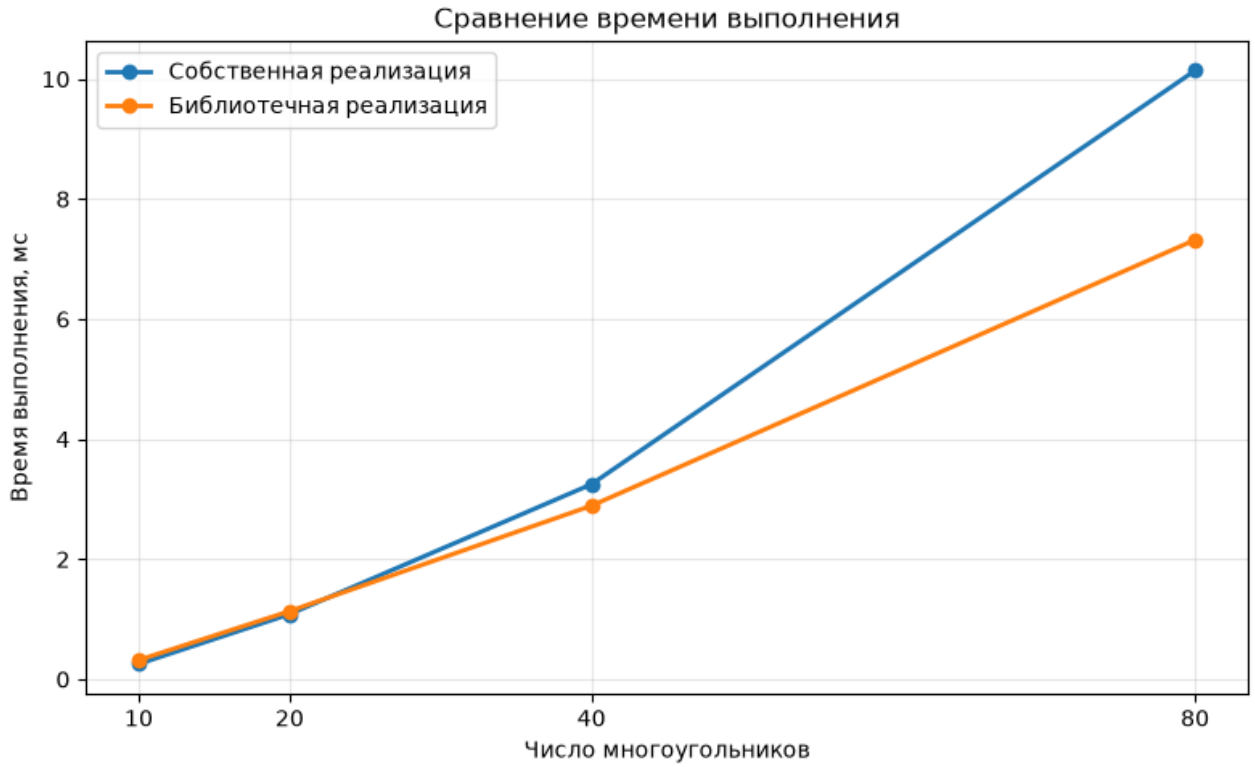


Рис. 9. Сравнение производительности реализаций

6.3 Анализ результатов

Полученные результаты показывают, что при небольшом количестве многоугольников времена выполнения собственной и библиотечной реализаций находятся в одном порядке. При увеличении размера сцены библиотечная реализация алгоритма работает быстрее, но не намного.

Таким образом, на использованном наборе тестов разработанный алгоритм корректно выполняет удаление невидимых линий и при этом показывает сопоставимую с библиотечной реализацией производительность.

7 Заключение

В ходе лабораторной работы был изучен и программно реализован алгоритм Вейлера–Айзертона для удаления невидимых линий в трёхмерной сцене, состоящей из многоугольников. Программа написана на языке C++. Она выполняет пересечение проекций многоугольников, определяет видимость фрагментов рёбер и позволяет визуализировать результат работы алгоритма.

Сравнение собственной реализации с библиотечной реализацией операции пересечения многоугольников показало совпадение результатов на тестовой сцене и позволило оценить производительность обоих подходов.